Documento técnico: NEXUS Expert

Sistema experto en informática con motor de inferencia por encadenamiento hacia adelante

Sistemas Inteligentes

Salomé Arroyave Urrea

Repositorio: https://github.com/sarroyaveu/sistema_experto

1. Introducción y objetivo

NEXUS Expert es un sistema experto construido a partir del motor de inferencia entregado en el taller. El código original tenía las reglas escritas dentro del programa, usaba una resolución de conflictos mínima y solo mostraba resultados por consola. El sistema desarrollado resuelve esas limitaciones: carga la base de conocimiento desde archivos externos, define una estrategia de resolución de conflictos con varios criterios, recibe los hechos iniciales por interacción o desde un archivo, ofrece una interfaz gráfica que muestra la inferencia paso a paso y entrega una recomendación al usuario.

Un único motor sirve a tres dominios de informática. Cada dominio tiene su propia base de conocimiento en JSON:

Dominio	El usuario indica	El sistema entrega
Ciberseguridad (27 reglas)	Las alertas que observa	Qué puede estar ocurriendo y qué debe hacer
Arquitectura en la nube (21 reglas)	Los requisitos de su sistema	La arquitectura recomendada
Modelos de IA (24 reglas)	Las necesidades y restricciones de su empresa	El modelo de IA que conviene desplegar

En total son 72 reglas, y cada dominio supera por mucho el mínimo de 15 reglas exigido.

2. Arquitectura del sistema

El sistema separa el conocimiento, el motor y la interfaz en tres capas independientes:

- **Base de conocimiento** (carpeta `knowledge_base/`): un archivo JSON por dominio con el catálogo de hechos y las reglas. Se puede reemplazar por otra base desde la interfaz sin tocar el código.
- **Motor de inferencia** (`engine.py`): carga y valida la base, ejecuta el encadenamiento hacia adelante, resuelve los conflictos y guarda la traza de cada paso. No sabe nada del dominio, así que el mismo código funciona para los tres.
- **Interfaz** (`app.py`, hecha con Streamlit): pantalla de inicio para elegir el dominio, ingreso de hechos mediante casillas agrupadas o carga de un archivo, ejecución del motor y presentación de resultados.

Flujo general: el usuario elige un dominio, la interfaz carga la base de conocimiento correspondiente, el usuario marca los hechos iniciales (o carga un caso de la carpeta `examples/`), el motor ejecuta la inferencia y la interfaz muestra la recomendación, los pasos y la cadena de razonamiento.

Carpetas del proyecto: `knowledge_base/` (conocimiento), `examples/` (casos de prueba con hechos iniciales), `docs/` (este documento), además de `app.py`, `engine.py`, `requirements.txt` y `README.md`.

3. Representación del conocimiento

3.1 Hechos

Todos los hechos se declaran en un catálogo dentro del JSON. Cada hecho tiene un identificador, un texto legible y un tipo:

- **entrada**: dato que el usuario puede marcar al ejecutar el sistema (un síntoma, una alerta, un requisito). Tiene además un campo `grupo` para organizar la interfaz.
- **intermedio**: hecho que el sistema deduce y que sirve de antecedente de otras reglas. Es lo que produce el encadenamiento.
- **recomendacion**: conclusión final que se muestra al usuario. Incluye un campo `detalle` con las acciones concretas a seguir.

La **memoria de trabajo** es un diccionario que asocia cada hecho con el paso en que apareció: los hechos iniciales tienen el paso 0 y los inferidos tienen el número del paso que los generó. Ese número se usa después en el criterio de recencia.

3.2 Reglas

Las reglas son producciones del tipo SI antecedentes ENTONCES consecuentes. Cada regla se escribe así:

- `id` y `nombre`: identifican la regla en la traza.
- `antecedentes`: lista de hechos que deben estar todos presentes (conjunción).
- `consecuentes`: hechos que se agregan al dispararse.
- `prioridad` (opcional): número entero que se usa como *salience*. Por defecto es 0.
- `excluye` (opcional): hechos que, si están presentes, impiden que la regla se dispare. Se usa en reglas de recomendación para que dos alternativas incompatibles no aparezcan juntas.

Al cargar la base, el motor valida que existan las claves obligatorias, que no haya identificadores duplicados y que todos los hechos usados en las reglas estén declarados en el catálogo. Si hay un error, la interfaz lo informa en lugar de fallar.

3.3 Hechos iniciales

Los hechos iniciales se pueden ingresar de dos maneras: marcando casillas en la interfaz, o cargando un archivo JSON. El archivo acepta tres formatos: un objeto con la clave `hechos`, una lista simple, o un diccionario `{hecho: true/false}` como el del código original. Los hechos que no existan en la base se ignoran y se avisa al usuario.

4. Estrategia de resolución de conflictos

En cada ciclo, el motor construye el **conjunto de conflictos**: las reglas que se podrían disparar en ese momento. Una regla entra al conjunto si cumple tres condiciones: no ha sido disparada antes, todos sus antecedentes están en la memoria de trabajo y ninguno de sus hechos `excluye` está presente. Además, debe aportar al menos un hecho nuevo, porque disparar una regla que no agrega nada no avanzaría la inferencia.

Luego se elige una sola regla aplicando los siguientes criterios en orden. Cada criterio solo se usa para desempatar el anterior:

1. **Refracción.** Una regla no se dispara dos veces ni entra al conjunto si no aporta hechos nuevos. Evita ciclos infinitos y trabajo repetido.
2. **Fases.** Mientras exista alguna regla de inferencia (que deduce hechos intermedios) aplicable, las reglas de recomendación no compiten. Las recomendaciones se evalúan solo cuando ya no queda nada por inferir, así que se emiten con toda la información derivada y no con una conclusión parcial.
3. **Prioridad explícita (*salience*).** Gana la regla con mayor valor de `prioridad`. Permite al experto del dominio indicar qué conclusiones son más urgentes (por ejemplo, el plan de respuesta ante un incidente crítico tiene prioridad 100).
4. **Recencia.** Gana la regla que usa hechos más recientes, comparando los pasos de aparición de sus antecedentes de mayor a menor. Favorece continuar la cadena de razonamiento que está en curso.
5. **Especificidad.** Gana la regla con más antecedentes, porque es más específica que una con menos condiciones.
6. **Orden de definición.** Si todavía hay empate, gana la que aparece primero en la base de conocimiento. Reemplaza el desempate al azar del enunciado y hace que el sistema sea determinista y reproducible, algo útil para explicar y probar.

Las recomendaciones se ordenan al final por prioridad y por el paso en que se generaron. La primera se muestra como recomendación principal y las demás como alternativas o complementos.

Referentes: esta estrategia se basa en las estrategias clásicas de los sistemas de producción: refracción, recencia y especificidad (estrategias LEX y MEA del lenguaje OPS5, descritas por Forgy, 1981, y analizadas por McDermott y Forgy, 1978); la prioridad explícita o *salience* que usa CLIPS (Giarratano y Riley, *Expert Systems: Principles and Programming*); y el encadenamiento hacia adelante descrito en Russell y Norvig, *Artificial Intelligence: A Modern Approach*.

5. Encadenamiento de reglas y trazabilidad

Las bases están diseñadas para que los consecuentes de unas reglas sean antecedentes de otras. Un ejemplo de ciberseguridad con una cadena de seis reglas:

- R01: correo sospechoso y clic en el enlace → phishing probable.
- R02: phishing probable → credenciales expuestas.
- R05: credenciales expuestas e inicio de sesión desde un país inusual → cuenta comprometida.
- R10: cuenta comprometida y tráfico saliente alto → exfiltración de datos.
- R16: exfiltración → incidente crítico.

- R18: incidente crítico → recomendar el plan de respuesta a incidentes.

En arquitectura en la nube, la cadena C04 → C05 → C14 lleva de tráfico variable y presupuesto limitado a optimizar costos, luego a serverless y finalmente a recomendar esa arquitectura. En modelos de IA, la cadena M01 → M03 → M04 → M16 lleva de generar texto a requerir un LLM, luego a un despliegue local por privacidad, luego a un modelo pequeño por poco hardware y finalmente a recomendar un LLM pequeño cuantizado.

El motor guarda, para cada hecho inferido, qué regla lo produjo y con qué antecedentes. Con ese registro la interfaz puede explicar por qué llegó a una conclusión, recorriendo la cadena hacia atrás hasta los datos iniciales. Esto es esencial en un sistema experto, porque el usuario puede verificar el razonamiento y no solo aceptar la respuesta.

6. Interfaz y resultados

La interfaz muestra, para cada ejecución:

- Métricas: hechos iniciales, hechos inferidos y reglas disparadas.
- La recomendación principal y las alternativas, con el detalle de las acciones.
- La inferencia paso a paso: en cada paso se ve el conjunto de conflictos con su prioridad, recencia y especificidad, la regla elegida, el motivo de la elección y los hechos nuevos. Un control deslizante permite avanzar paso por paso.
- La cadena de razonamiento de cada recomendación.
- Un botón para descargar el resultado en JSON.

7. Decisiones de diseño

- **JSON como formato de conocimiento.** Es legible, se edita sin programar, se puede versionar en Git y Python lo lee sin dependencias.
- **Un motor para tres dominios.** Separar el conocimiento del código demuestra que el motor es genérico y que agregar un dominio nuevo solo requiere un archivo nuevo.
- **Tipos de hecho.** Distinguir entrada, intermedio y recomendación permite construir la interfaz de forma automática y separar las fases de inferencia y recomendación.
- **Fase de recomendación separada.** Evita que una recomendación aparezca antes de tiempo, cuando todavía podrían derivarse conclusiones más graves.
- **Campo `excluye`.** Permite que el sistema recomiende una sola opción cuando dos son incompatibles, por ejemplo contenedores gestionados en una región frente a Kubernetes multi-región.
- **Desempate determinista.** Se reemplazó el azar por el orden de definición para que los resultados sean reproducibles.

8. Limitaciones

- Los hechos son verdaderos o ausentes. No se manejan grados de certeza ni probabilidades.
- Las reglas se conectan solo con conjunciones. Las disyunciones se expresan escribiendo varias reglas con el mismo consecuente.

- Las recomendaciones son orientativas y se basan en el conocimiento codificado en las reglas. No reemplazan el criterio de un especialista en seguridad, arquitectura o ciencia de datos.

9. Uso de herramientas de IA generativa

Se declara que en este trabajo se utilizó una herramienta de IA generativa: **Claude**. Se redactar el código del motor de inferencia, de la interfaz y de las bases de conocimiento, sugerir la estrategia de resolución de conflictos y redactar la primera versión de este documento. Los integrantes revisaron, probaron y ajustaron los resultados, y conocen el funcionamiento completo del sistema para la sustentación.

De igual forma se usó GEMINI para poner en palabras más concretas la idea que se tenía para que posteriormente chat gpt con esa idea más clara generara un prompt para que claude generara algunas cosas que se le pidieron.

Referencias

- McDermott, J. y Forgy, C. (1978). Production system conflict resolution strategies. En *Pattern-Directed Inference Systems*. Academic Press.
- Forgy, C. L. (1981). *OPS5 User's Manual*. Carnegie Mellon University.
- Giarratano, J. y Riley, G. *Expert Systems: Principles and Programming*. Cengage.
- Russell, S. y Norvig, P. *Artificial Intelligence: A Modern Approach*. Pearson.